



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 11

Problem Statement

In this assignment, you are provided with `chess_engine.py` and a set of chess piece images. The current implementation provides a graphical interface using Pygame and move logic via the `python-chess` library, but the AI simply selects moves at random.

Your task is to replace the random move generator with an intelligent AI using “Alpha-Beta Pruning” and a “Heuristic Evaluation” function. The final program must:

1. Allow the user to choose a color (White or Black) at startup via the terminal.
2. Render the board and pieces graphically, ensuring the board is oriented correctly for the player.
3. Implement an AI that:
 - i. Uses the Minimax algorithm with Alpha-Beta pruning to a depth of at least 3.
 - ii. Evaluates board states based on material balance and basic positional advantages (e.g., piece-square tables).
4. Handle full game logic, including turns and terminal states (Checkmate, Stalemate, or Draw).

Algorithm Specifications

1. Heuristic Evaluation Function:

- Create a function `evaluate_board(board)` that returns a numerical score.
- Use a standard convention: a positive score favors White, and a negative score favors Black.
- Consider material weights: Pawn (100), Knight/Bishop (320/330), Rook (500), Queen (900).

2. Alpha-Beta Pruning:

- Implement `minimax(board, depth, alpha, beta, maximizing_player)`.
- Ensure the algorithm prunes branches that cannot influence the final decision to improve search efficiency.
- **Note:** Move ordering (evaluating captures first) can significantly improve pruning performance.

3. Function Specification:

- Implement `get_best_move(board, depth)` which initializes the alpha-beta search and returns the optimal `chess.Move` object.
- The AI must account for whether it is playing as White (maximizing the score) or Black (minimizing the score).

4. Game Flow and Technical Notes:

- On the human turn, capture mouse clicks to generate moves. **Important:** When moving a pawn to the 8th rank, remember to set the `promotion` attribute of the `chess.Move` object (e.g., to `chess.QUEEN`).
- The AI turn should call your search function. A depth of 3 is recommended for a balance between speed and performance in Python.

Documentation and Move Representation

The engine utilizes the `python-chess` library. Refer to the documentation for advanced logic:

<https://python-chess.readthedocs.io/en/latest/>

Key Library Components:

1. `board.legal_moves`: A dynamic container of all valid moves for the current side.
2. `board.push(move) / board.pop()`: Used to simulate and undo moves during the search tree traversal.
3. `board.is_checkmate()`: Returns `True` if the current player is in checkmate.

Universal Chess Interface (UCI) Notation

Moves are represented using UCI strings (e.g., `e2e4`).

- **Standard Move:** A 4-character string representing the source and destination squares (e.g., `g1f3`).
- **Promotion:** A 5th character is added for the promoted piece (e.g., `a7a8q` for a Queen promotion).
- **Conversion:** Use `move.uci()` to get the string or `chess.Move.from_uci("e2e4")` to create a move object.